

Competitive Programming

Section 43: String Algorithms I

Competitive Programming Course

2026-09-13

Table of contents

1	Prefix-function thinking	3
1.1	The update rule	3
2	KMP matching	5
3	Borders and periods	6
3.1	Every border, largest first	6
3.2	The shortest period	6
4	Implementation, linearity, and debugging	8
4.1	Why it is linear	8
4.2	Language notes	8
4.3	Tests before trusting a solution	8
5	Exercises	10

1 Prefix-function thinking

One array does all the work in this section. For a string s , the prefix function is

```
pi[i] = length of the longest proper prefix of s[0..i]
        that is also a suffix of s[0..i]
```

Proper means it cannot be the whole of $s[0..i]$, so $pi[i] < i+1$ always. Storing a length is enough; the two copies are equal by definition, so there is never a reason to copy the substring itself.

The running example throughout this section is **abacaba**, and the four questions the section answers are: what is the longest border of each prefix (problem D's ingredient), where does a pattern occur in a text (problem A), what are *all* the borders (problem B), and what is the shortest period (problem C).

i	0	1	2	3	4	5	6
s	a	b	a	c	a	b	a
π	0	0	1	0	1	2	3

a	b	a	c	a	b	a	$\pi[2] = 1$ border a
a	b	a	c	a	b	a	$\pi[5] = 2$ border ab
a	b	a	c	a	b	a	$\pi[6] = 3$ border aba

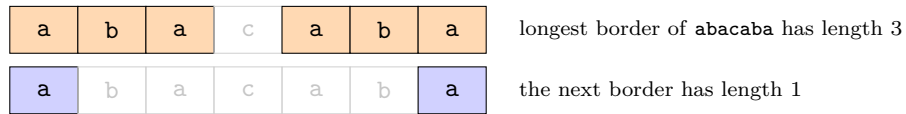
prefix copy and **suffix copy** must not overlap the whole string: the border is *proper*, so it is always shorter than $s[0..i]$.
 Faded cells lie beyond position i and are not part of the prefix being measured.
 π stores only the *length*; the two copies are equal by definition.

1.1 The update rule

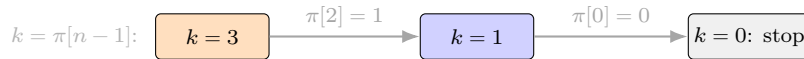
Positions are filled left to right. To extend to $s[i]$, take the best border of the previous prefix, $j = pi[i-1]$, and try to grow it by one character:

```
j = pi[i-1]
while j > 0 and s[i] != s[j]:
    j = pi[j-1]
if s[i] == s[j]:
    j += 1
pi[i] = j
```

The **while** loop is the only subtle line, and it is not a search. If a border of length j is contradicted, every longer border was already ruled out, so the longest candidate still alive is the longest border *of that border* — which is $\pi[j-1]$, a value already computed.



A border of a border is again a border, so the border lengths form a chain that π already stores. Following it enumerates *every* proper border, largest first — that is exactly problem B.



The same chain is the failure link. While building π , suppose the candidate border has length j and the new character contradicts it. Every border longer than $\pi[j-1]$ has already been ruled out, so the longest still-possible candidate is $\pi[j-1]$ — never 0, and never a fresh restart. In **abacaba** at $i = 3$ the candidate is $j = \pi[2] = 1$; since $s[3] = c \neq s[1] = b$ it falls to $\pi[0] = 0$, and $c \neq a$ leaves $\pi[3] = 0$.

2 KMP matching

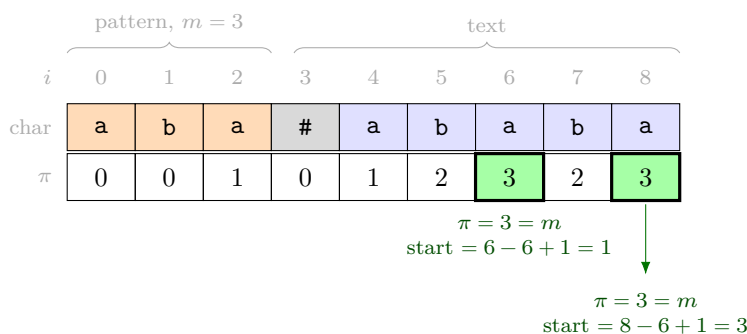
To find every occurrence of a pattern p of length m in a text t , run the prefix function once over

```
p + separator + t
```

The separator must be a character that cannot appear in either string. Its only job is to stop a prefix from stretching across the join, which would report matches that do not exist. If the alphabet is not under your control, do not concatenate at all; run the matching loop directly against p and t with the same fallback rule.

Wherever $\pi[i] == m$, the pattern has just finished. Counting back over the pattern, the separator, and the matched copy gives the 1-based start in the text:

```
start = i - 2*m + 1
```



Why overlapping matches are not lost. At $i = 7$, just after a full match, the candidate is $j = 3$. The next character b contradicts it, so the loop falls to $\pi[2] = 1$ — the longest border of the pattern — and b matches there, giving $\pi[7] = 2$. One more character reaches 3 again at $i = 8$. A naive scanner that restarted at 0 after a match would skip the occurrence at position 3.

Why the text index never retreats. Only j moves backwards, and it never falls below 0 while i only advances. Each character raises j by at most one, so total fallback is bounded by total increase: $O(n)$.

Overlapping occurrences come out for free, and that is the whole advantage over naive scanning: after a match the algorithm falls back to the longest border of the pattern, never to zero, so characters already known to match are never compared again.

3 Borders and periods

3.1 Every border, largest first

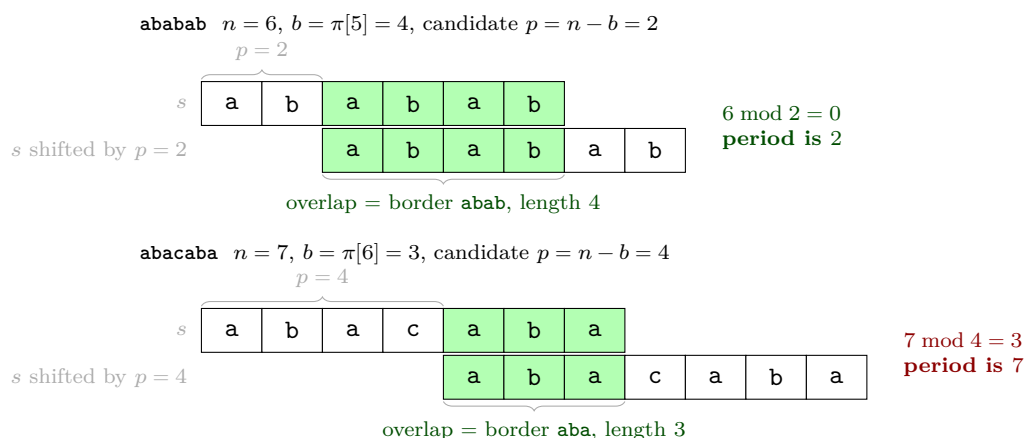
The chain in the figure above is also the answer to problem B. The longest border is $\text{pi}[n-1]$; the next is the longest border of *that* prefix, and so on until the chain reaches zero:

```
k = pi[n-1]
while k > 0:
    record k
    k = pi[k-1]
```

This produces borders in decreasing order. Reverse the list if the statement asks for increasing order. For **abacaba** it yields 3, 1; for **abcbaca** it yields 4, 1; for **abc** it yields nothing at all.

3.2 The shortest period

A border of length b says the string agrees with itself after a shift of $p = n - b$. That shift is always a valid *overlap* statement, but a period that tiles the string needs one extra condition:



A border of length b says the string matches itself after a shift of $p = n - b$, so $s[i] = s[i + p]$ wherever both exist. That makes p the shortest *candidate* period, and it is always correct as an overlap statement.

The trap. It is a *tiling* period only when p divides n . For **abacaba** the shift is consistent, but 4 does not divide 7: the string is not whole copies of **abac**, so the answer is $n = 7$. Always test $n \bmod p$ before printing p .

```
b = pi[n-1]
p = n - b
answer = p if n % p == 0 else n
```

ababa is the smallest trap worth memorizing: $\text{pi}[4] = 3$, so the candidate is $5 - 3 = 2$, but 2 does not divide 5 and the answer is 5.

4 Implementation, linearity, and debugging

4.1 Why it is linear

The index `i` never moves backwards. The pointer `j` rises by at most one per position, so across the whole run it rises at most `n` times. Every iteration of the `while` loop strictly lowers `j`, and `j` never drops below zero, so the total number of fallback steps is bounded by the total number of increases. Both are $O(n)$, which is why a loop that looks quadratic is not.

The loop invariant behind that argument: before position `i` is processed, `j` is the longest border of `s[0..i-1]` not yet contradicted by the new character. Each fallback preserves the longest surviving candidate; each success extends it by exactly one.

4.2 Language notes

C++. A `vector<int>` for `pi` is enough. Reserve the combined string once rather than appending in a loop. Occurrence counts can exceed `int` on adversarial inputs such as `aaaa...a`, so accumulate in `long long`.

Python. Read with `sys.stdin.buffer.read()` and work on `bytes`; indexing a `bytes` object yields integers and compares faster than slicing strings. Never build the combined string by repeated `+=` in a loop.

4.3 Tests before trusting a solution

Case	What it catches
pattern longer than text	negative or out-of-range start positions
pattern equals text	off-by-one in <code>i - 2*m + 1</code>
<code>aba</code> in <code>ababa</code>	a scanner that restarts at zero and loses overlaps
<code>aaaa</code> in <code>aaaaa</code>	the maximal-overlap stress case
no occurrence at all	an empty-output path
<code>ababa</code> for the period	the missing <code>n % p == 0</code> divisibility test
<code>abc</code> for borders	an empty border chain
separator present in input	a bogus match across the join

Useful values to check by hand: `pi("aaaaa") = 0 1 2 3 4`, the proper borders of `abcabca` are 1 and 4, and `abacaba` has candidate period 4 but answer 7.

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/43_string_algorithms_i/check.py
```

5 Exercises

Use A to get the combined-string indexing and the 1-based start formula right, B to walk the border chain to exhaustion rather than stopping at the first link, and C to apply the divisibility test that turns a candidate period into a real one. Use D as the blind synthesis problem: every prefix needs a count, not just the whole string, so decide what a failure link contributes to its target before writing any code. Test a pattern longer than the text, a pattern equal to the text, overlapping matches, an all-same-character input, and a string with no border at all.

- A. KMP Occurrences: Print the number and 1-based positions of all occurrences of pattern in text.
- B. All Borders: Print all proper border lengths of a string in increasing order.
- C. Minimal Period: Print the shortest period length if the string is repetitions of a block, otherwise `n`.
- D. Prefix Frequency: For every prefix length, print how many times that prefix occurs as a substring.